

part__1

October 25, 2025

0.0.1 Data Preprocessing, training ML models and Artificial Neural Networks (ANN) with GridSearchCV

1. 1.Import Libraries
 2. 1.1 Loading the dataset
 3. 2.Data Preprocessing
 4. 2.1 Separate for training and test split
 5. 2.2 Select categorical, numerical and continuous features
 6. 2.3 impute NA values with different methods
 7. 2.4 Pipeline, ColumnTransformer, find best parameters impute NA values with GridSearchCV
 8. 2.5 Temporary pipeline. Select Mean Imputer for further preprocessing
 9. 2.6 Outliers handling
 10. 2.7 Discretization with EqualFrequencyDiscretiser
 11. 2.8 Discretization with EqualWidthDiscretiser
 12. 2.9 DecisionTreeDiscretiser
 13. 3.Training ML Models with default parameters
 14. 3.1 Preprocessing Pipeline for ML models
 15. 3.2 Separate for training and test split to train ANN
 16. 3.3 Preprocessing Pipeline for ANN
 17. 3.4 Early stopping
 18. 3.5 Build the ANN
 19. 3.6 GridSearchCV
 20. 4.Results
 21. 5.Conclusion
1. Import Libraries

```
[9]: # Import basic libraries
import pandas as pd
import numpy as np

# library for show graphs
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats

# library for separate to train and test split
from sklearn.model_selection import train_test_split, KFold
```

```

# library for impute missing value
from feature_engine.imputation import MeanMedianImputer, RandomSampleImputer,
↳EndTailImputer, CategoricalImputer

# library for encode and scaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import RobustScaler, MinMaxScaler, MaxAbsScaler,
↳StandardScaler
from category_encoders.woe import WOEEncoder
from feature_engine.encoding import MeanEncoder, CountFrequencyEncoder,
↳WoEEncoder
from feature_engine.encoding import OneHotEncoder as fe_OneHot

# ANN library
import tensorflow as tf
from scikeras.wrappers import KerasRegressor
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout, BatchNormalization
from keras.optimizers import Adam, SGD, Nadam, RMSprop, Adadelta
from keras.callbacks import EarlyStopping
from scikeras.wrappers import KerasClassifier
from tensorflow.keras.regularizers import l2
from tensorflow.keras import regularizers

#Algorithms
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier,
↳BaggingClassifier, ExtraTreesClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import SGDClassifier

from sklearn.compose import ColumnTransformer

# Callback, Checkpoint
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping,
↳ReduceLROnPlateau, LearningRateScheduler

# metrics score
from sklearn.metrics import classification_report, confusion_matrix,
↳roc_auc_score, f1_score, recall_score, accuracy_score, precision_score

```

```

from sklearn.model_selection import cross_val_score, cross_val_predict

# GridSearchCV
from sklearn.model_selection import GridSearchCV

# Discretization
from feature_engine.discretisation import EqualFrequencyDiscretiser,
↳EqualWidthDiscretiser, DecisionTreeDiscretiser

#Outliers
from feature_engine.outliers import Winsorizer

# feature selection
from feature_engine.selection import (
    DropConstantFeatures,
    DropDuplicateFeatures,
    SmartCorrelatedSelection
)

#Pipeline
from sklearn.pipeline import Pipeline

pd.pandas.set_option('display.max_columns', None)

```

1.1 Loading the dataset

```

[10]: dataset = pd.read_csv('/Users/shabdantaalaibekov/Documents/KV_AI_JLM_7/
↳Human_Resources_TP.csv')
dataset.head()

```

2. Data Preprocessing

```

[11]: dataset['Attrition'] = dataset['Attrition'].apply(lambda x: 1 if x == 'Yes'
↳else 0)
dataset['OverTime'] = dataset['OverTime'].apply(lambda x: 1 if x == 'Yes' else
↳0)
dataset['Gender'] = dataset['Gender'].apply(lambda x: 1 if x == 'Female' else
↳0) # 0 Male 1 Female

```

```

[12]: print(dataset.Attrition.unique())
print(dataset.OverTime.unique())
print(dataset.Gender.unique())

```

```

[13]: # Drop constant feat
dataset.drop(labels = ['EmployeeCount', 'StandardHours', 'Over18',
↳'EmployeeNumber'], axis = 1, inplace = True)

```

2.1 Separate for training and test split

```
[14]: X_train, X_test, y_train, y_test = train_test_split(
        dataset.drop(labels = ['Attrition'], axis = 1),
        dataset['Attrition'],
        test_size = 0.25
    )
    X_train.shape, X_test.shape
```

2.2 Select categorical, numerical and continuous features

```
[15]: categorical_feat = [
        col for col in dataset.columns if dataset[col].dtype == 'O' and col != 'Attrition'
    ]

    numerical_feat = [
        col for col in dataset.columns if dataset[col].dtype != 'O' and col != 'Attrition'
    ]
    len(categorical_feat), len(numerical_feat)
```

```
[16]: continuous = [
        col for col in dataset.columns if dataset[col].dtype == float and col != 'Attrition'
    ]
    len(continuous)
```

```
[17]: def find_skewed_boundaries(df, distance):
        skewed_variables = []
        for column in df.columns:
            if df[column].dtype != 'object': #
                IQR = df[column].quantile(0.75) - df[column].quantile(0.25)
                lower_boundary = df[column].quantile(0.25) - (IQR * distance)
                upper_boundary = df[column].quantile(0.75) + (IQR * distance)
                if (df[column] < lower_boundary).any() or (df[column] >
                    upper_boundary).any():
                    skewed_variables.append((column, lower_boundary,
                    upper_boundary))
        return skewed_variables
```

```
[18]: find_skewed_boundaries(dataset, 1.5)
```

```
[19]: X_train = X_train.copy()
    X_test = X_test.copy()
```

```
[20]: def diagnostic_plots(dataset, variable):

        plt.figure(figsize = (16, 4))
```

```

plt.subplot(1, 3, 1)
sns.histplot(dataset[variable], bins = 30)
plt.title('Histogram')

plt.subplot(1, 3, 2)
stats.probplot(dataset[variable], dist = 'norm', plot = plt)
plt.ylabel('Variable Quantile')

plt.subplot(1, 3, 3)
sns.boxplot(y = dataset[variable])
plt.title('Box-Plot')

plt.show()

```

```

[21]: def plot_before_after_imp(before_set, after_set, variable_names):
        fig, axes = plt.subplots(nrows=len(variable_names), ncols=1, figsize=(12, 5*
        ↪ len(variable_names)))

        if len(variable_names) == 1:
            axes = [axes]

        for ax, var in zip(axes, variable_names):
            ax.set_title(f'Distribution Before and After Scaling: {var}')
            sns.kdeplot(before_set[var], ax=ax, label='Before Scaling',
            ↪ color='blue')
            sns.kdeplot(after_set[var], ax=ax, label='After Scaling', color='red')
            ax.legend()

        plt.tight_layout()
        plt.show()

variable_names = X_train.select_dtypes(include=[np.number]).columns

```

2.3 impute NA values with different methods

```

[22]: # Random Sample Imputer for numerical variables
random_imp = RandomSampleImputer(variables=numerical_feat, random_state=42)
random_X_train = random_imp.fit_transform(X_train.copy())
random_X_test = random_imp.transform(X_test.copy())

# Mean Imputer for numerical variables
mean_imp = MeanMedianImputer(imputation_method='mean', variables=numerical_feat)
mean_X_train = mean_imp.fit_transform(X_train.copy())
mean_X_test = mean_imp.transform(X_test.copy())

# Median Imputer for numerical variables

```

```

median_imp = MeanMedianImputer(imputation_method='median',
    ↪variables=numerical_feat)
median_X_train = median_imp.fit_transform(X_train.copy())
median_X_test = median_imp.transform(X_test.copy())

# EndTail Imputer for numerical variables
endTail_imp = EndTailImputer(imputation_method='gaussian', tail='right',
    ↪variables=numerical_feat)
end_X_train = endTail_imp.fit_transform(X_train.copy())
end_X_test = endTail_imp.transform(X_test.copy())

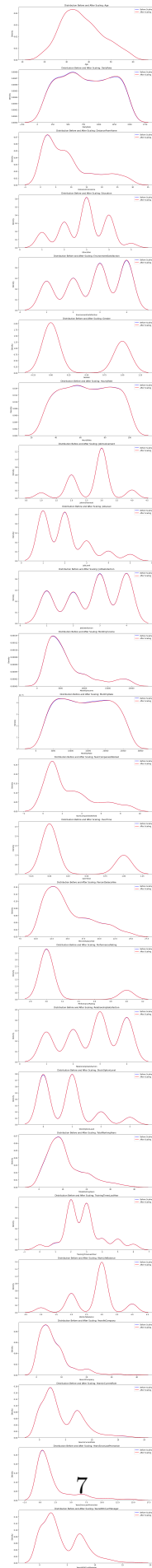
# Categorical Imputer for categorical variables
categorical_imp = CategoricalImputer(imputation_method='missing',
    ↪variables=categorical_feat)
cat_X_train = categorical_imp.fit_transform(X_train.copy())
cat_X_test = categorical_imp.transform(X_test.copy())

```

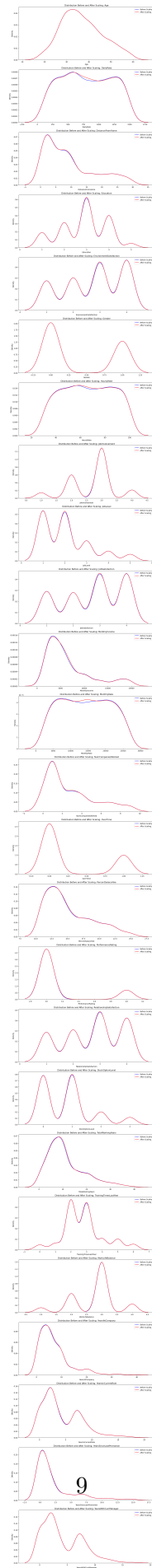
```

[23]: # random imp
plot_before_after_imp(X_train, random_X_train, variable_names)

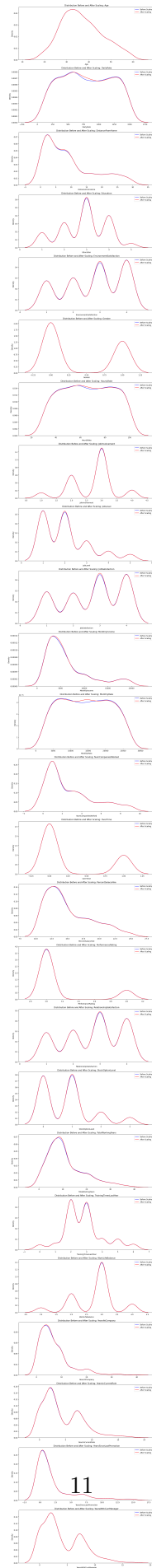
```



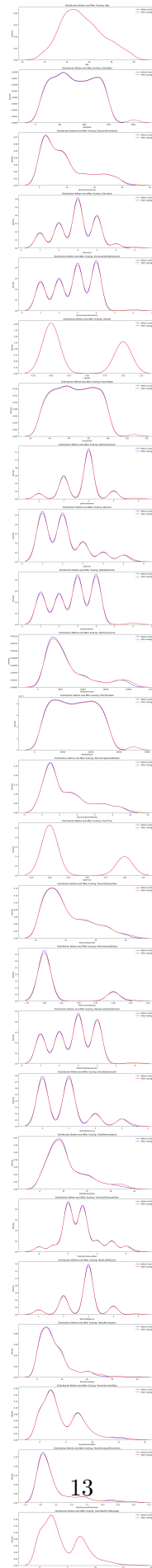
```
[24]: # mean imp  
      plot_before_after_imp(X_train, mean_X_train, variable_names)
```



```
[25]: # median imp  
      plot_before_after_imp(X_train, median_X_train, variable_names)
```



```
[26]: # end-tail imp  
      plot_before_after_imp(X_train, end_X_train, variable_names)
```



2.4 Pipeline, ColumnTransformer, find best parameters impute NA values with GridSearchCV

```
[27]: # pipeline for imputation missing values of categorical variables
categorical_pipeline = Pipeline(steps=[
    ('imputer', CategoricalImputer(variables=categorical_feat,
    ↪imputation_method='frequent')),
    ('onehot', fe_OneHot(variables=categorical_feat))
])

# pipeline for imputation missing values of numerical variables with different
    ↪methods
numerical_transformers = {
    'random': RandomSampleImputer(variables=numerical_feat, random_state=42),
    'mean': MeanMedianImputer(imputation_method='mean',
    ↪variables=numerical_feat),
    'median': MeanMedianImputer(imputation_method='median',
    ↪variables=numerical_feat),
    'endTail': EndTailImputer(imputation_method='gaussian', tail='right',
    ↪variables=numerical_feat)
}

# ColumnTransformer for concatenate
preprocessors = {
    name: ColumnTransformer(transformers=[
        ('numerical', transformer, numerical_feat),
        ('categorical', categorical_pipeline, categorical_feat)
    ]) for name, transformer in numerical_transformers.items()
}

# pipeline with different preprocessor
pipelines = {
    name: Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('classifier', RandomForestClassifier(random_state=42))
    ]) for name, preprocessor in preprocessors.items()
}

# find best params with GridSearchCV
param_grid = {
    'random': {
        'preprocessor__numerical__seed': ['general'],
        'preprocessor__numerical__seeding_method': ['add', 'multiply'],
        'classifier__n_estimators': [100, 200, 250]
    },

```

```

    'mean': {
        'preprocessor__numerical__imputation_method': ['mean', 'median'],
        'classifier__n_estimators': [100, 200, 250]
    },
    'median': {
        'preprocessor__numerical__imputation_method': ['mean', 'median'],
        'classifier__n_estimators': [100, 200, 250]
    },
    'endTail': {
        'preprocessor__numerical__tail': ['right', 'left'],
        'preprocessor__numerical__imputation_method': ['gaussian', 'iqr',
↪ 'max'],
        'preprocessor__numerical__fold': [1.5, 2, 3],
        'classifier__n_estimators': [100, 200, 250]
    }
}
# Create list of all pipelines and parameters for GridSearchCV
all_pipelines = [(name, pipeline) for name, pipeline in pipelines.items()]
all_params = [(name, params) for name, params in param_grid.items()]

# Create GridSearchCV for each pipeline
grid_searches = {
    name: GridSearchCV(estimator=pipeline, param_grid=params, cv=10, n_jobs=-1,
↪ error_score='raise')
    for (name, pipeline), (_, params) in zip(all_pipelines, all_params)
}

# train GridSearchCV for each pipeline and results
results = []

for name, grid_search in grid_searches.items():
    print()
    print(f"Running GridSearchCV for {name} pipeline...")
    grid_search.fit(X_train, y_train)
    best_score = grid_search.best_score_
    results.append((name, best_score))
    print()
    print(f"Best Parameters for {name} pipeline:", grid_search.best_params_)
    print()

sorted_results = sorted(results, key = lambda x: x[1], reverse = True)

for name, best_score in sorted_results:
    print(f"Best Score for {name} pipeline: {best_score}")

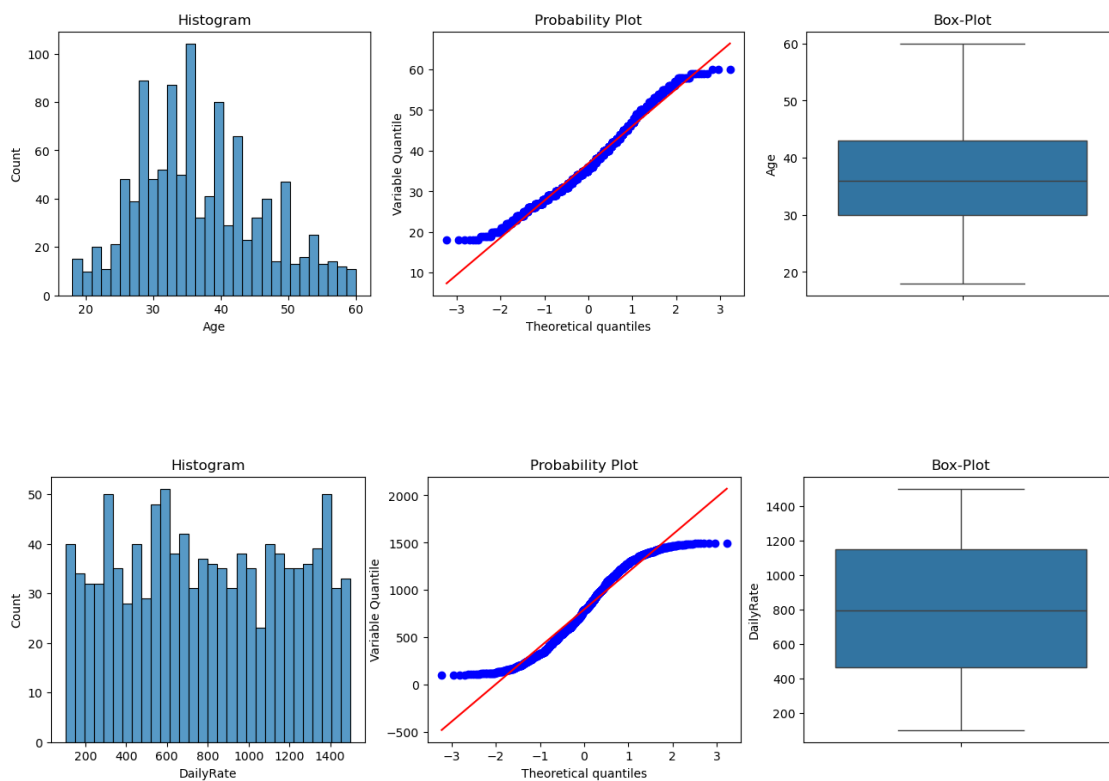
```

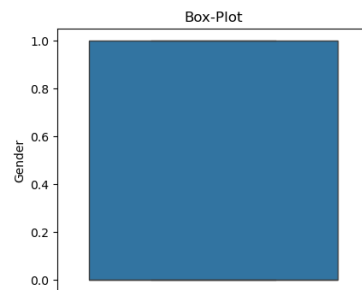
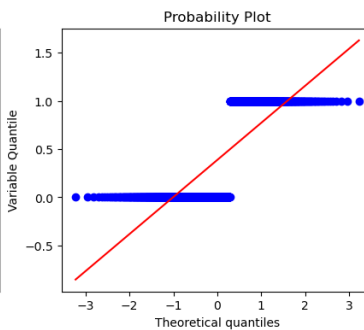
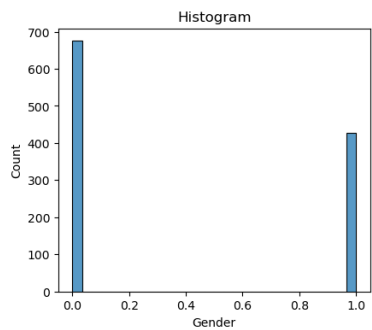
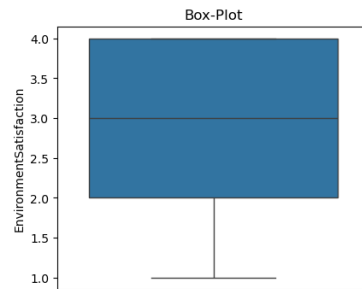
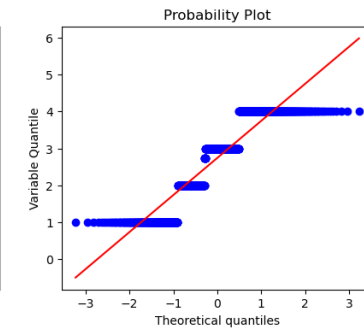
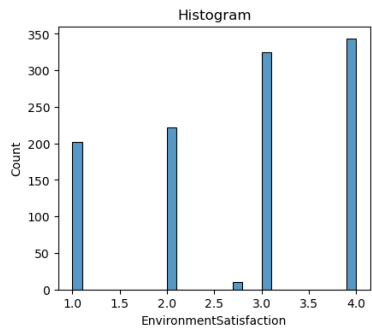
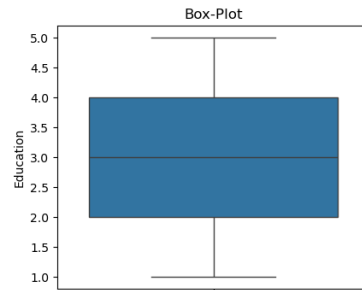
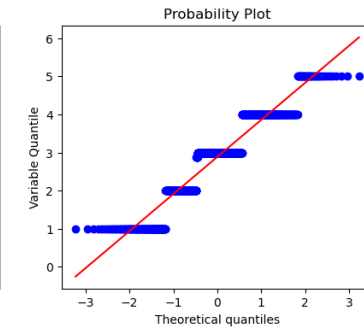
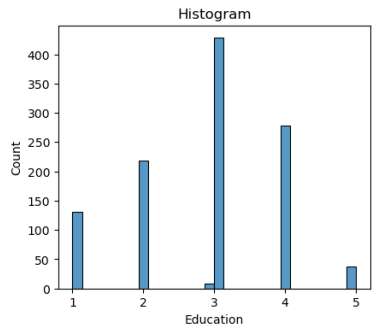
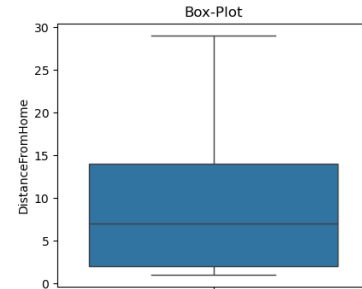
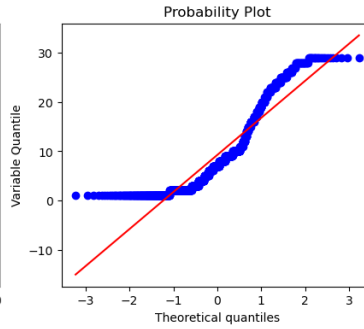
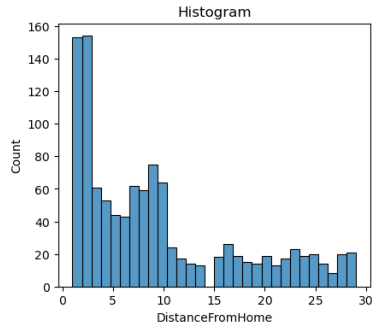
2.5 Temporary pipeline

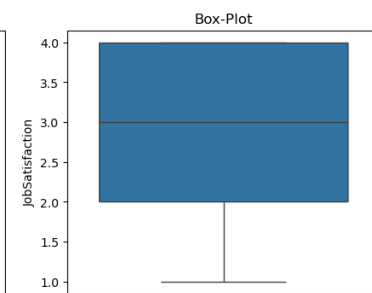
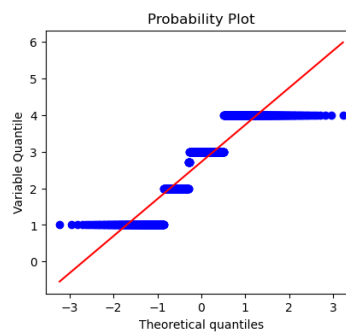
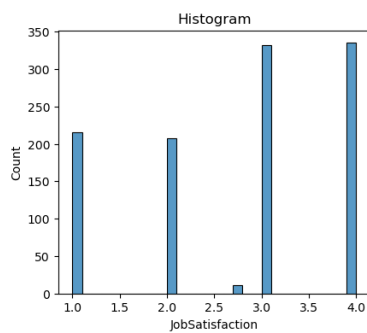
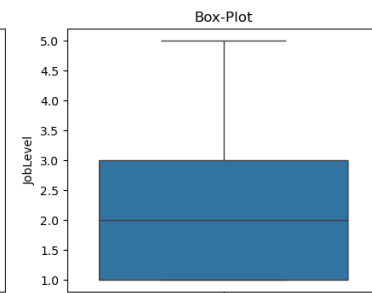
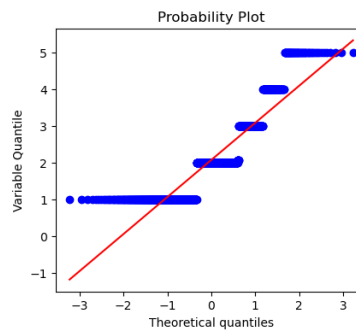
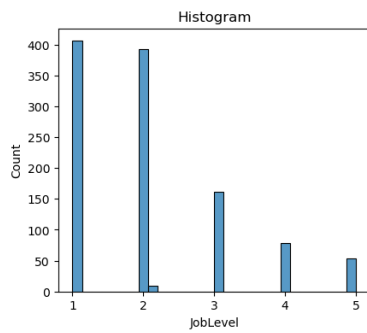
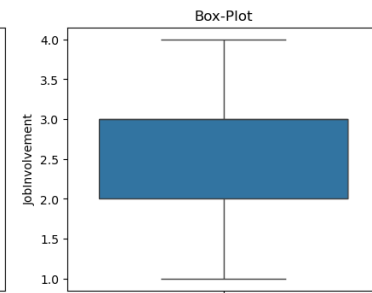
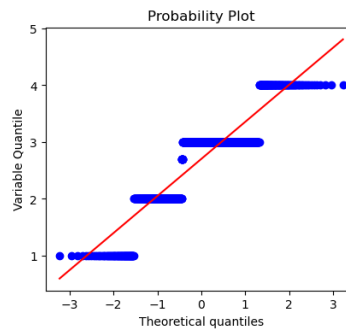
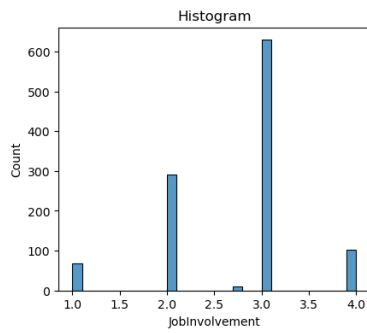
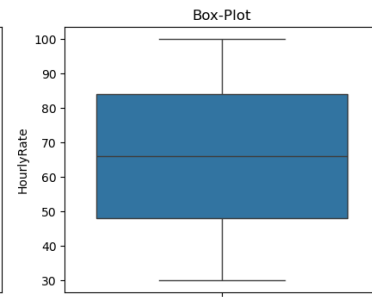
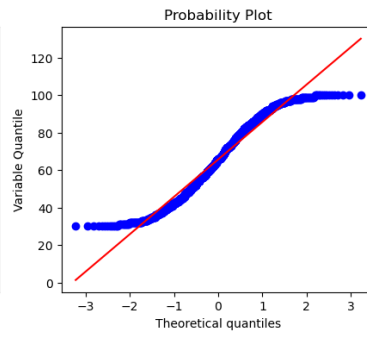
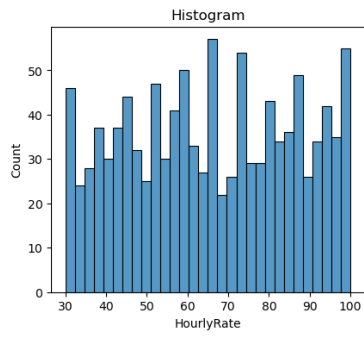
```
[28]: pipe_mean = Pipeline(steps = [
    ('imp_num', MeanMedianImputer(variables = numerical_feat, imputation_method_
↪ = 'mean')),
    ('imp_cat', CategoricalImputer(variables = categorical_feat)),
    ('ohe', fe_OneHot(variables = categorical_feat))
])
pipe_mean.fit(X_train)

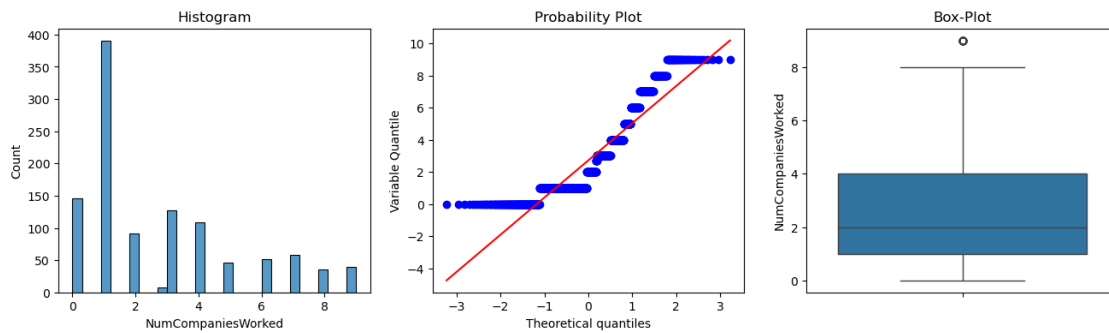
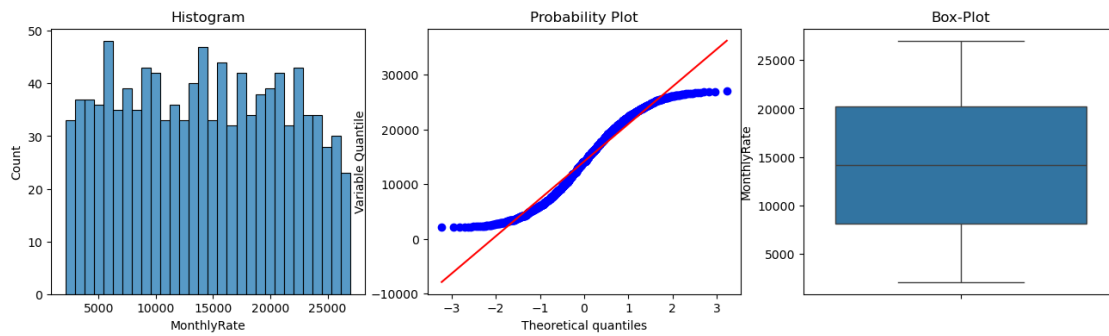
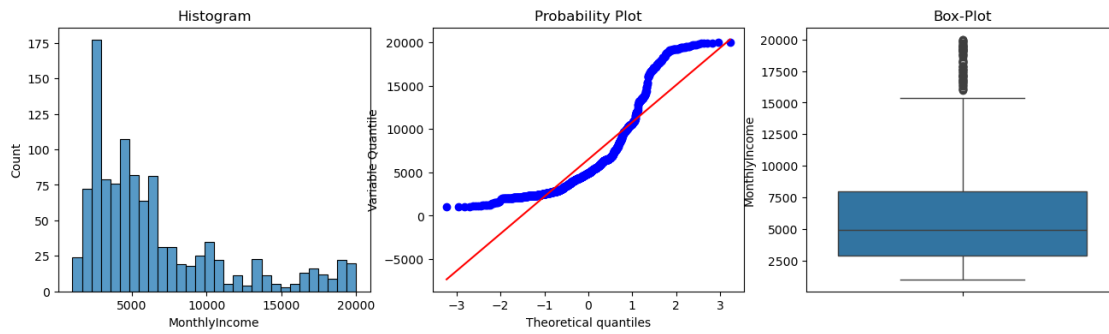
tmp_mean_train = pipe_mean.transform(X_train)
tmp_mean_test = pipe_mean.transform(X_test)
```

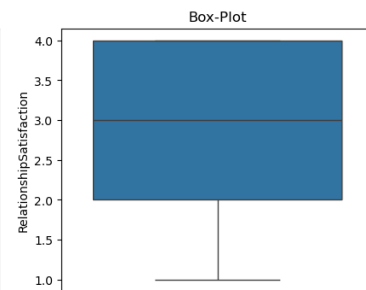
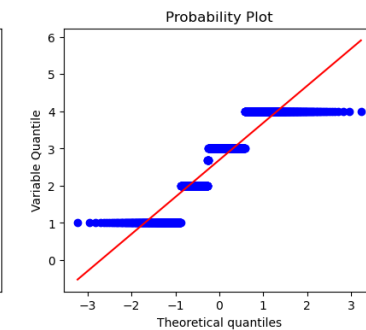
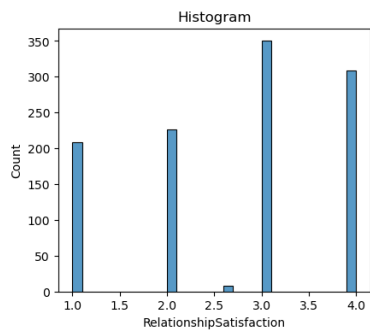
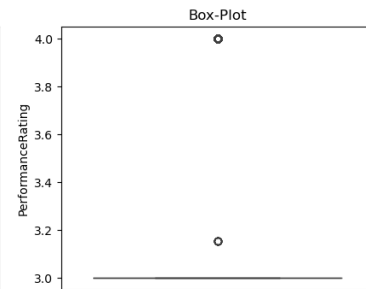
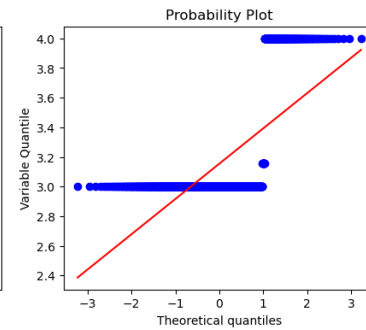
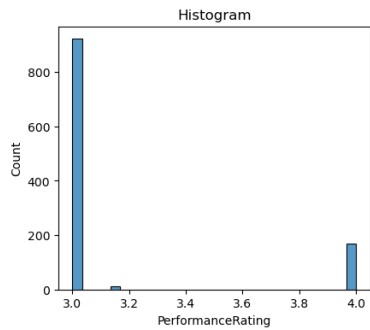
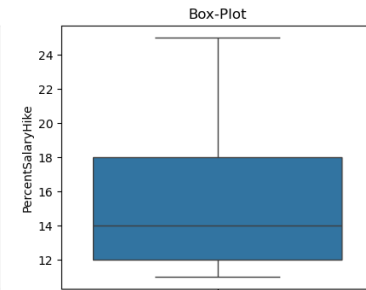
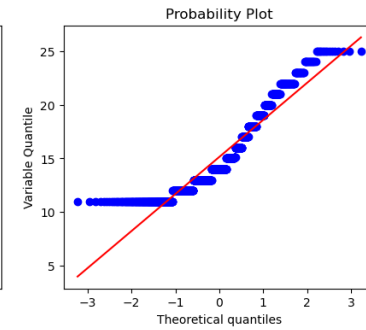
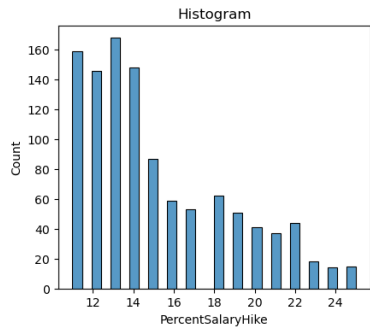
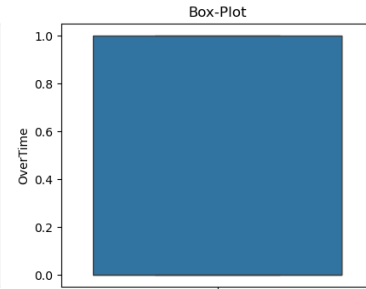
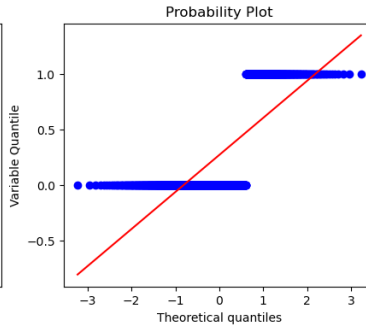
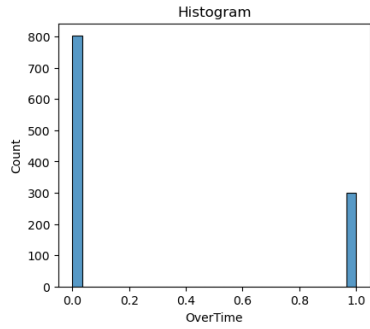
```
[29]: for col in tmp_mean_train.columns:
    diagnostic_plots(tmp_mean_train, col)
```

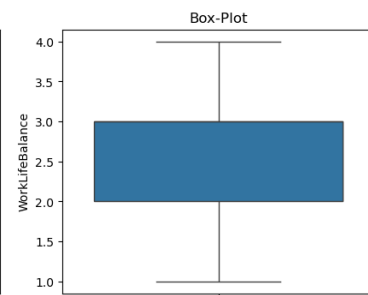
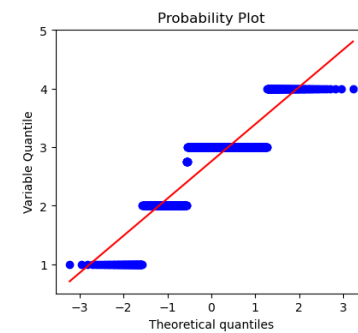
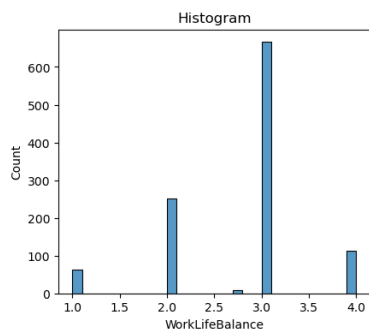
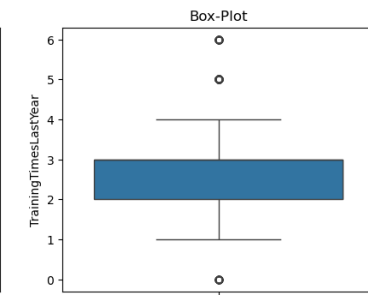
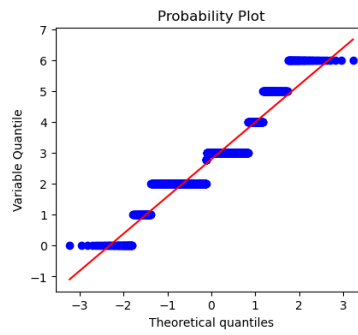
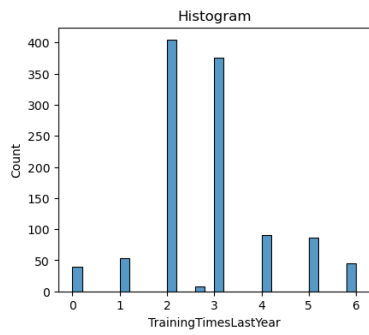
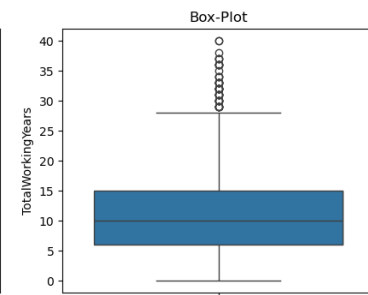
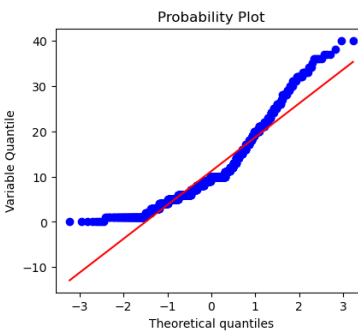
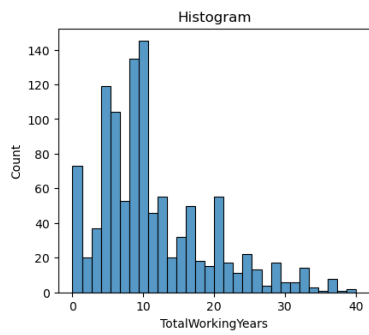
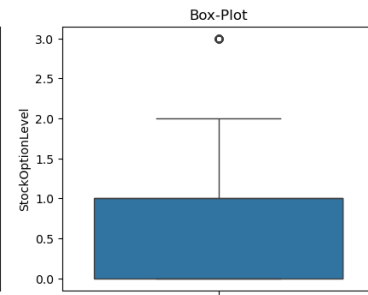
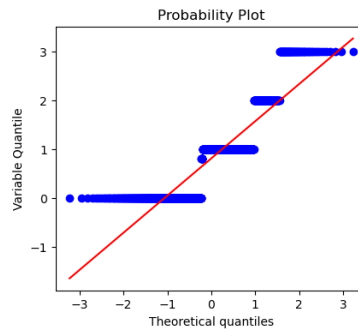
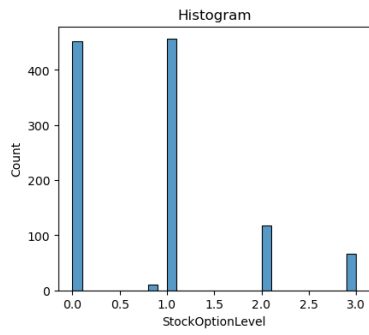


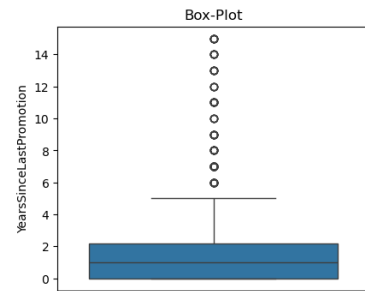
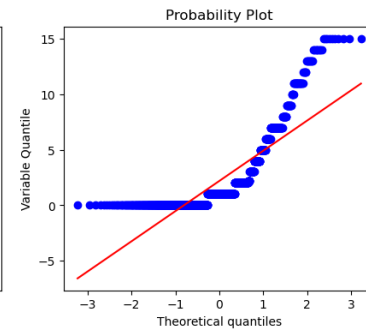
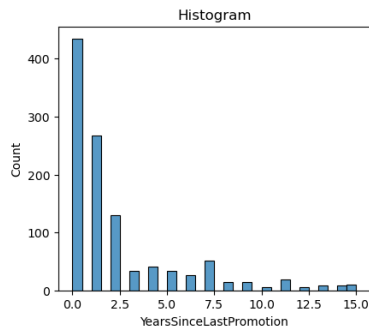
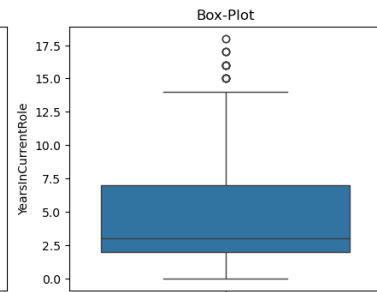
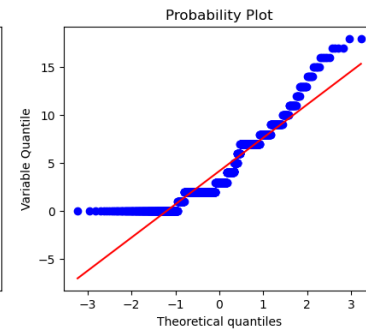
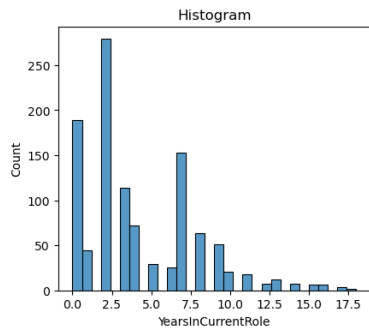
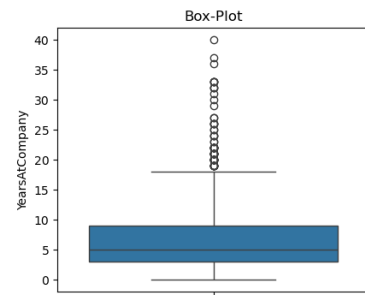
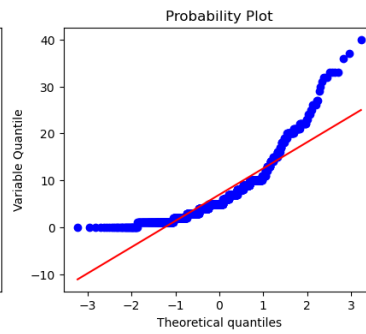
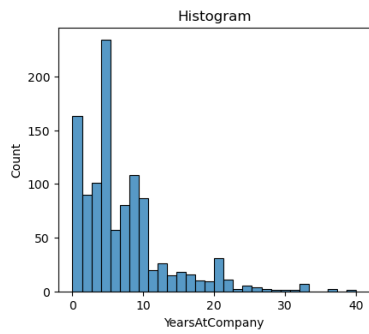


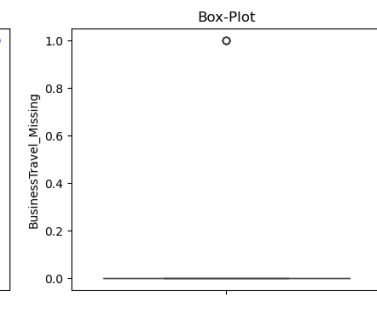
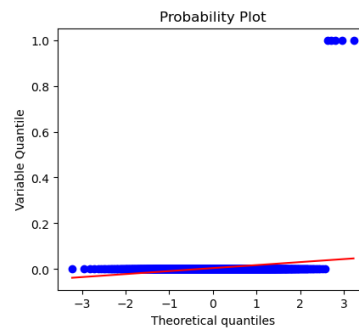
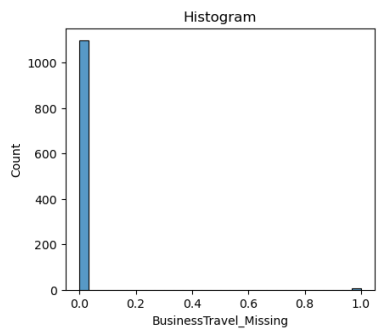
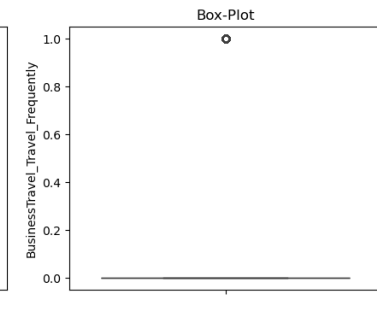
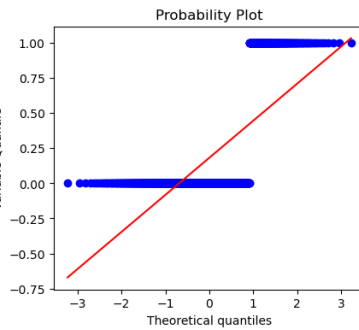
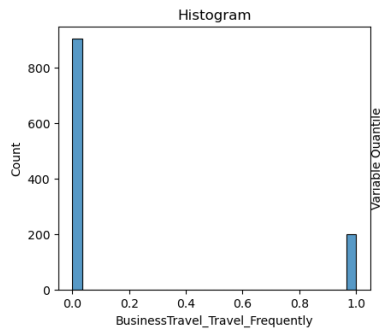
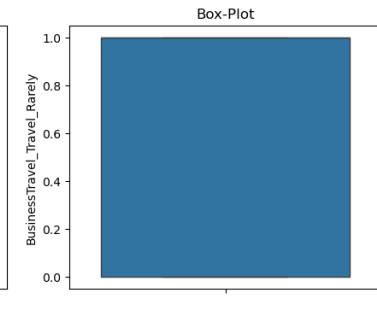
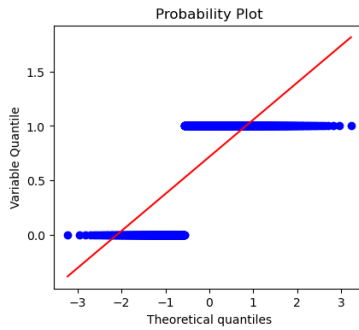
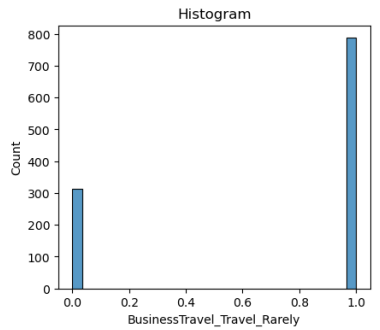
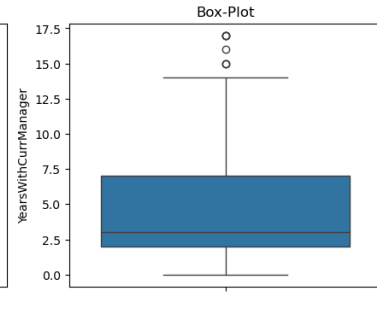
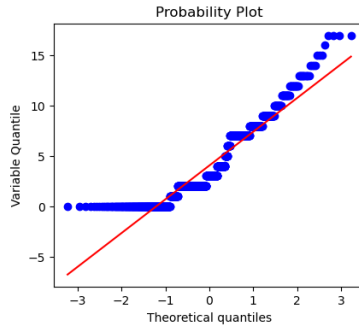
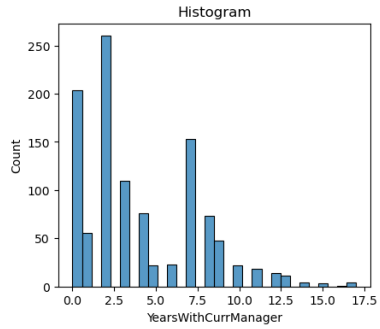


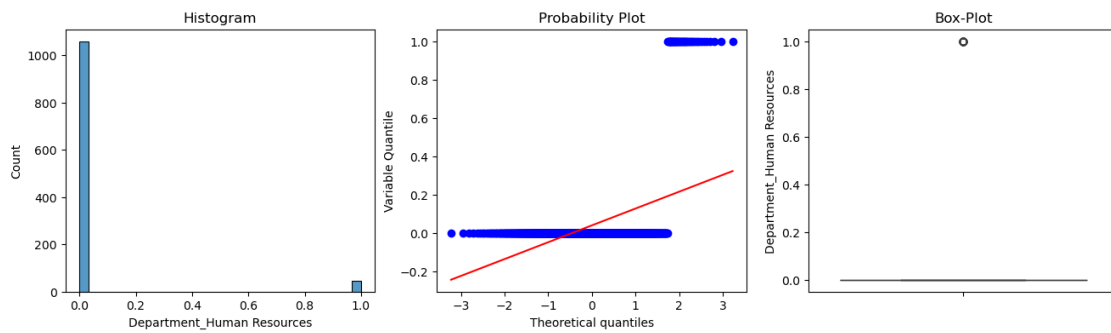
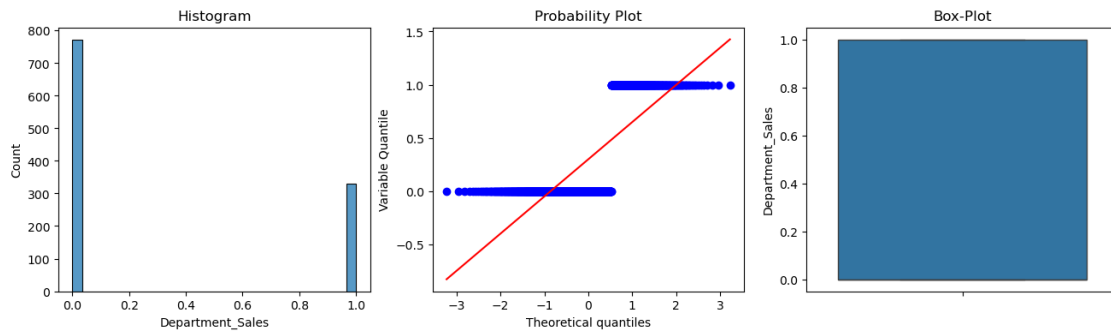
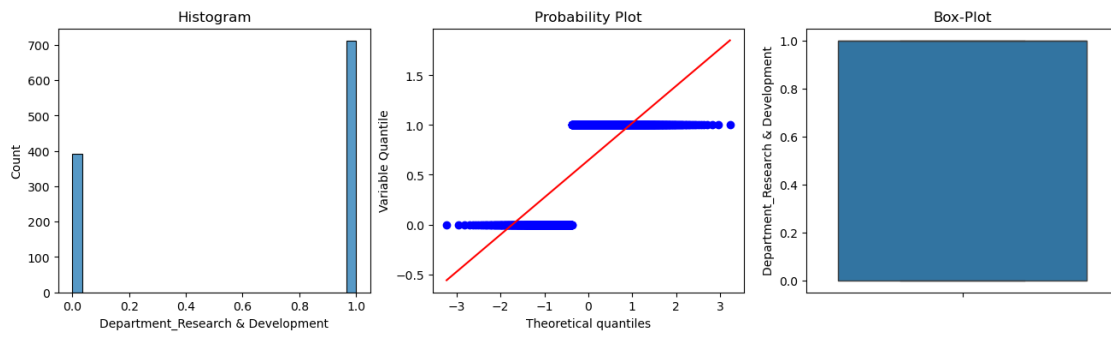
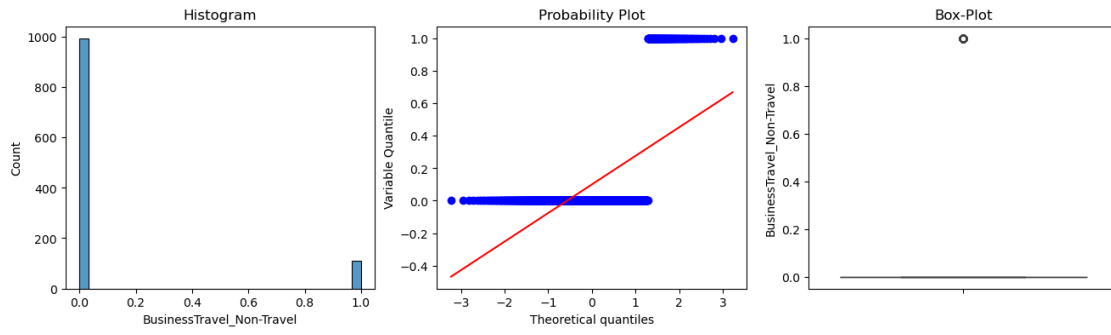


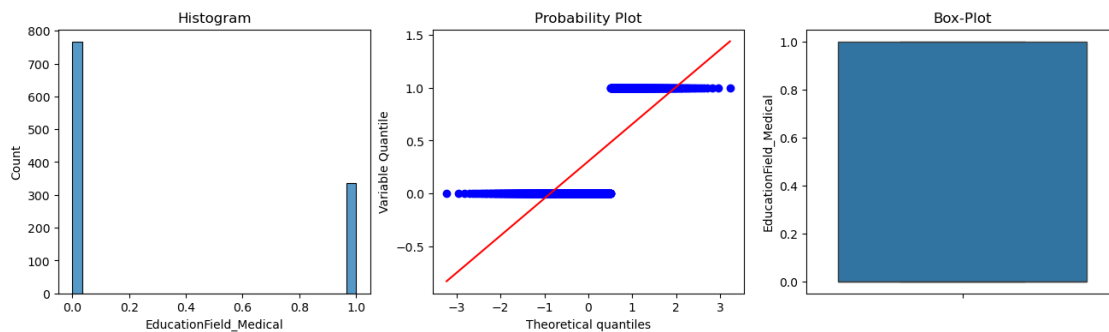
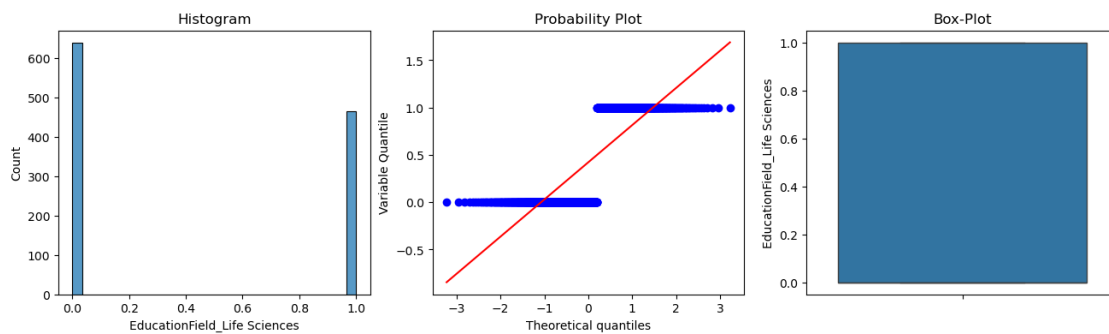
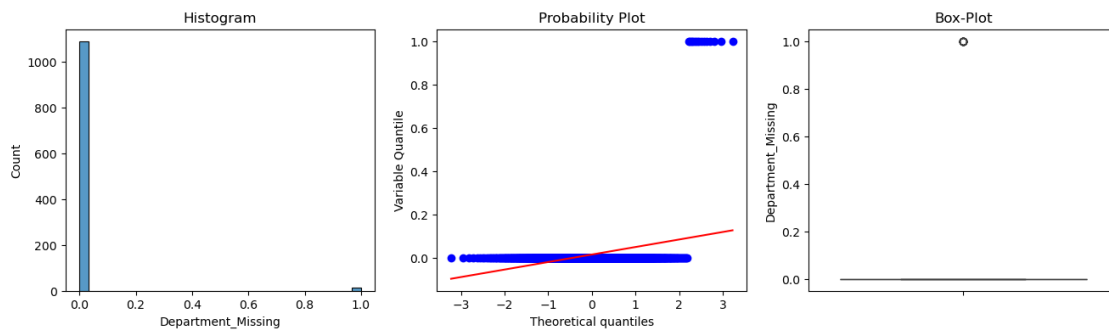


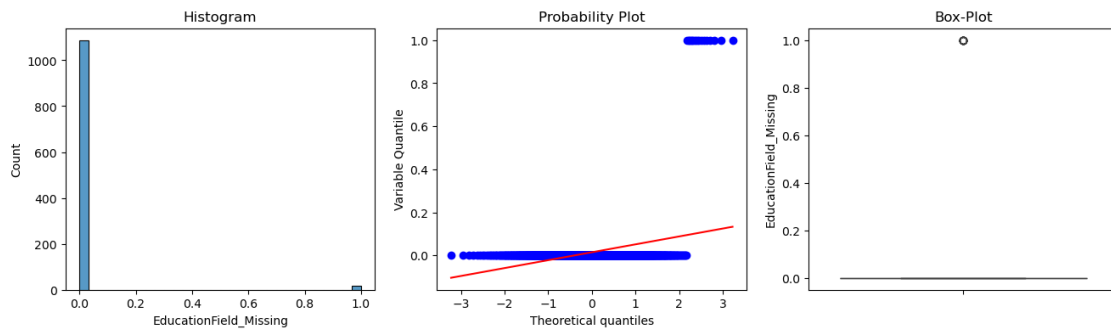
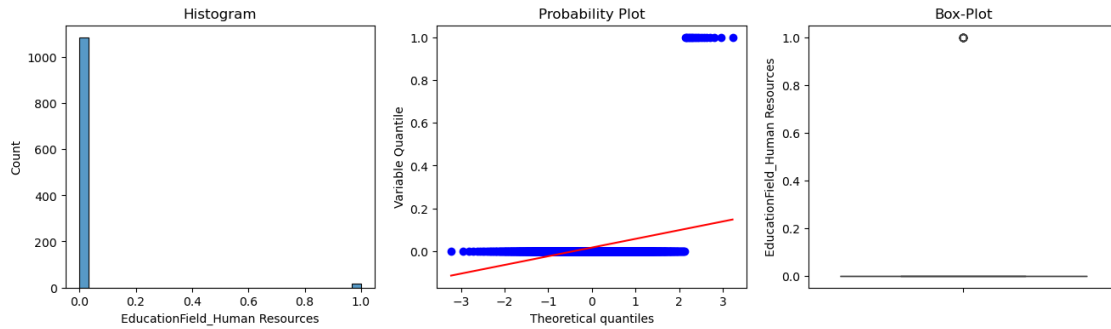
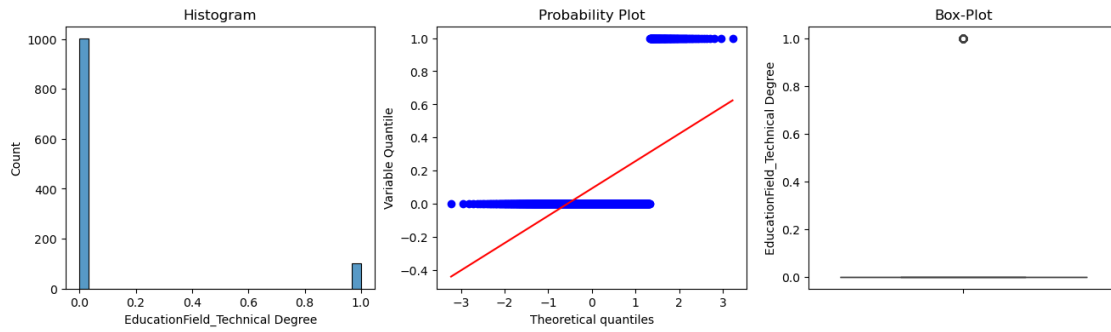
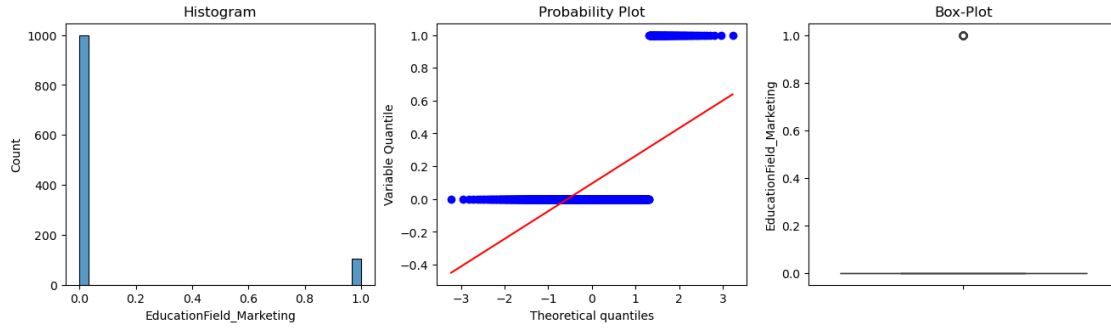


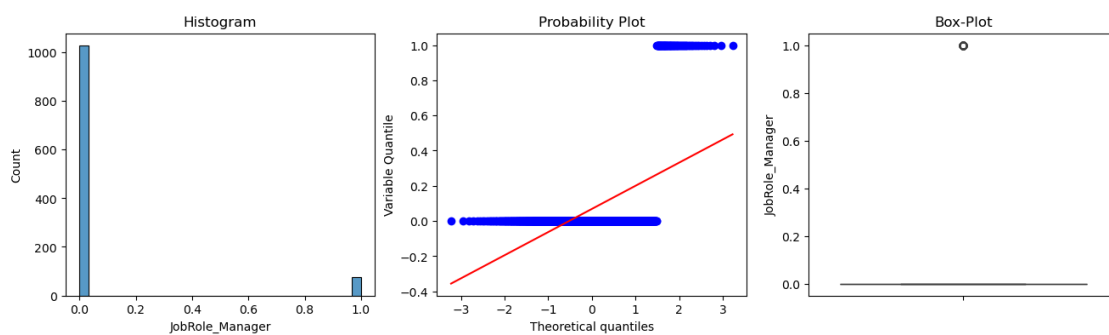
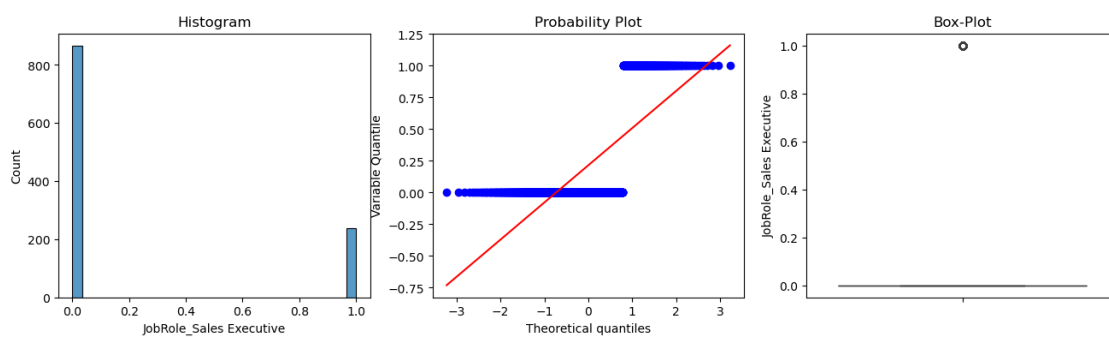
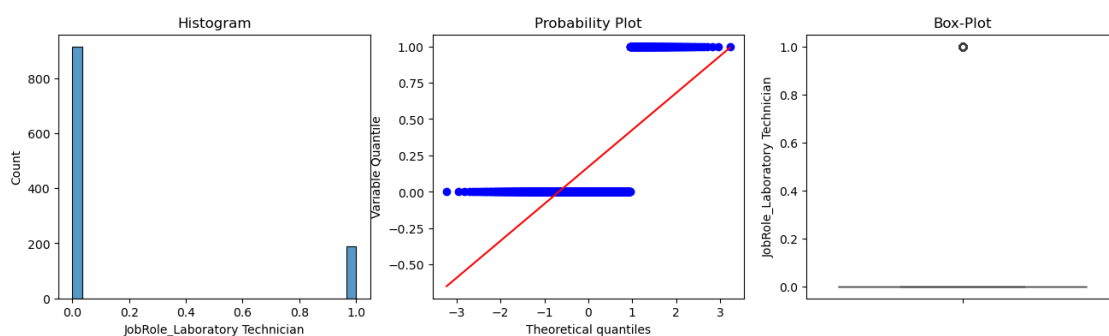
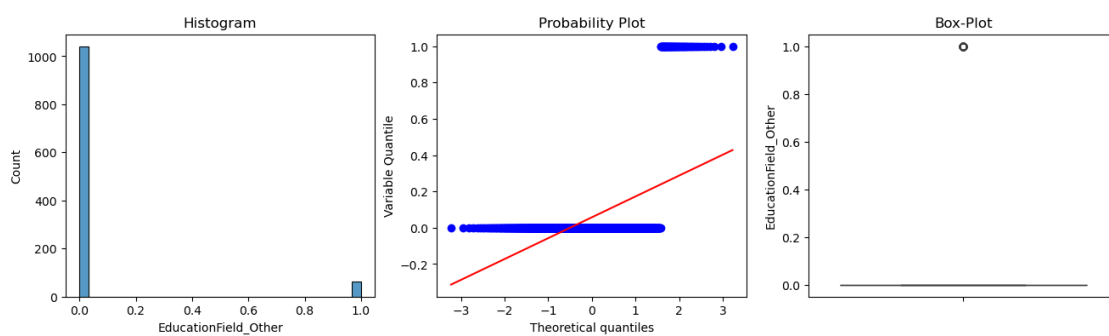


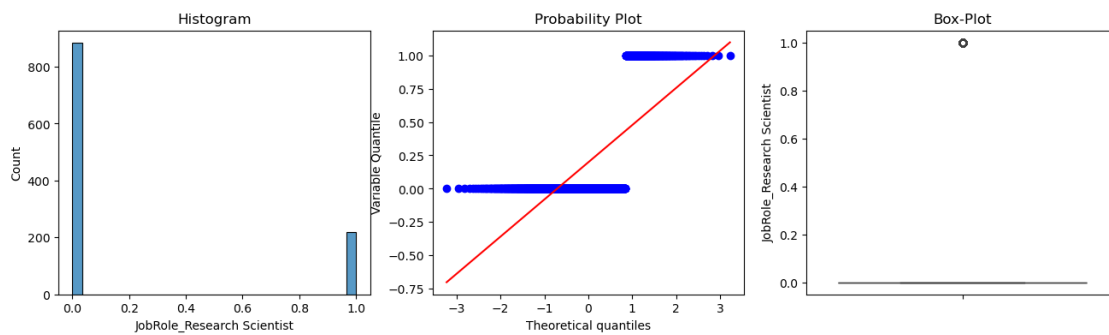
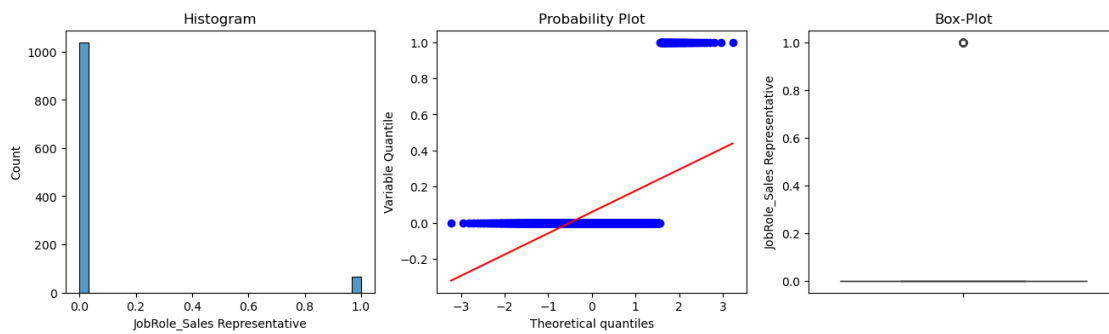
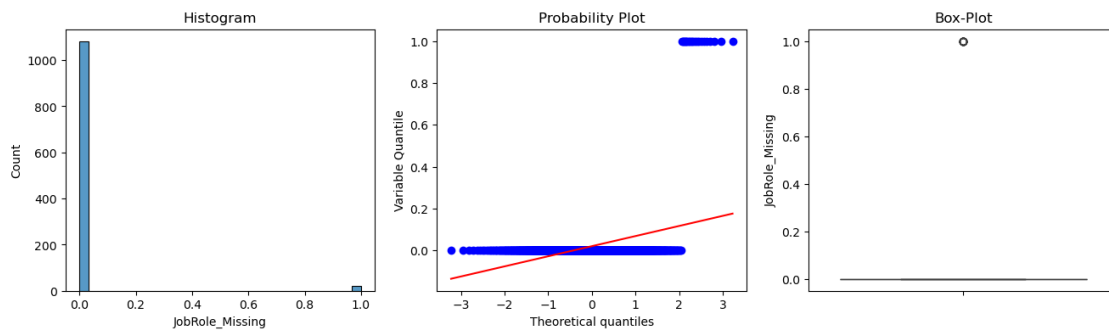


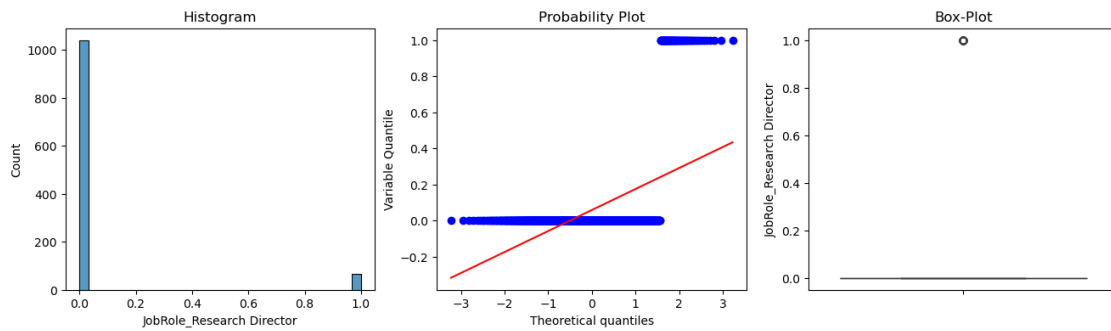
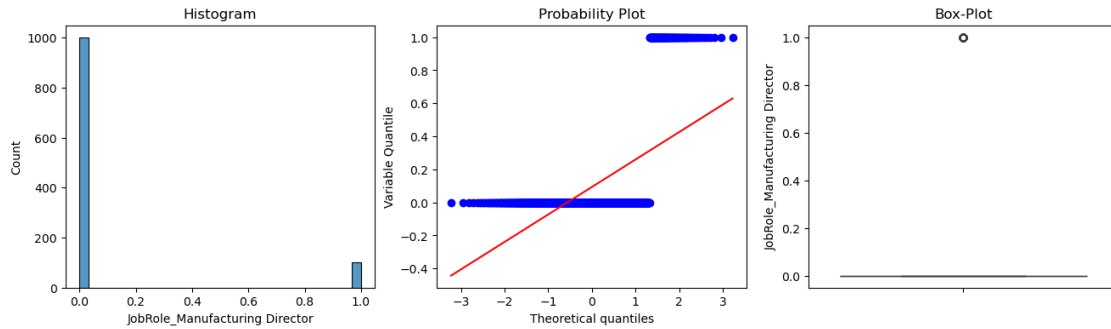
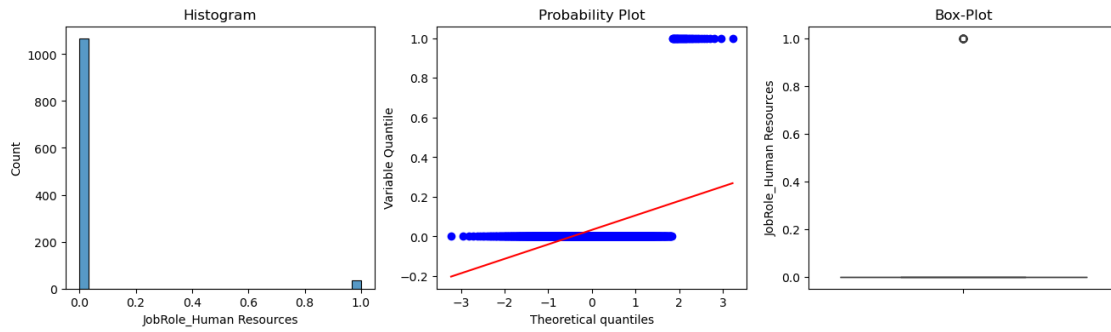
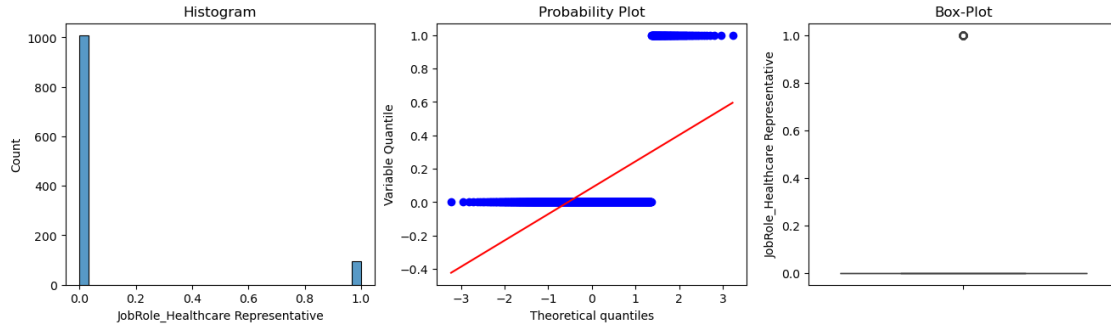


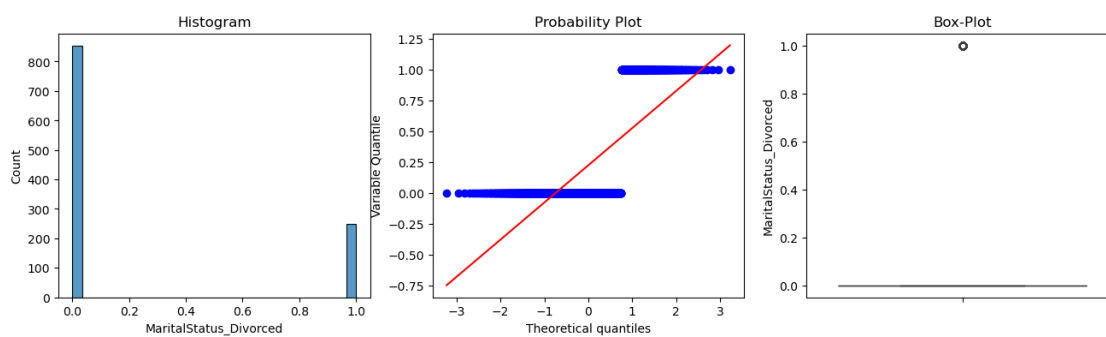
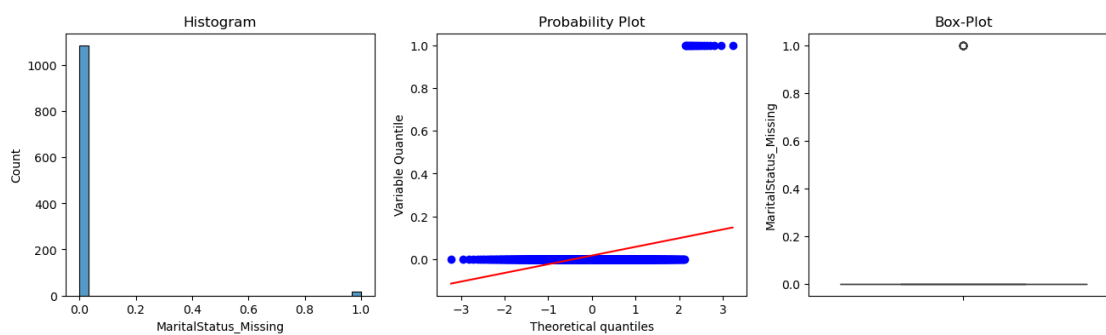
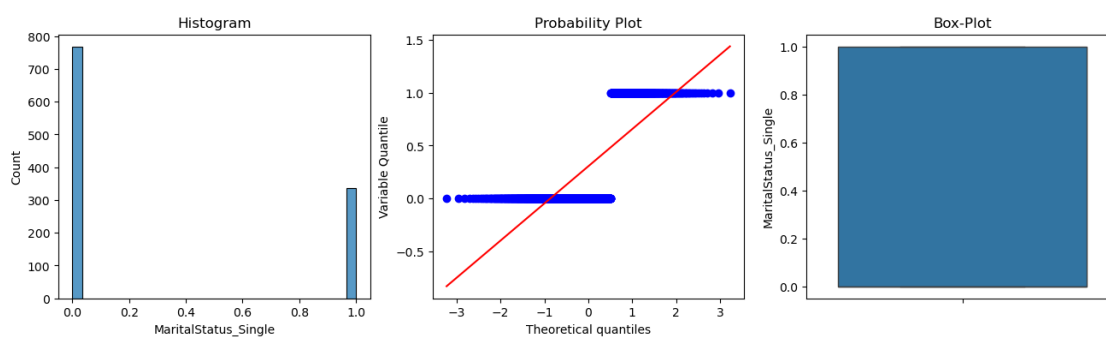
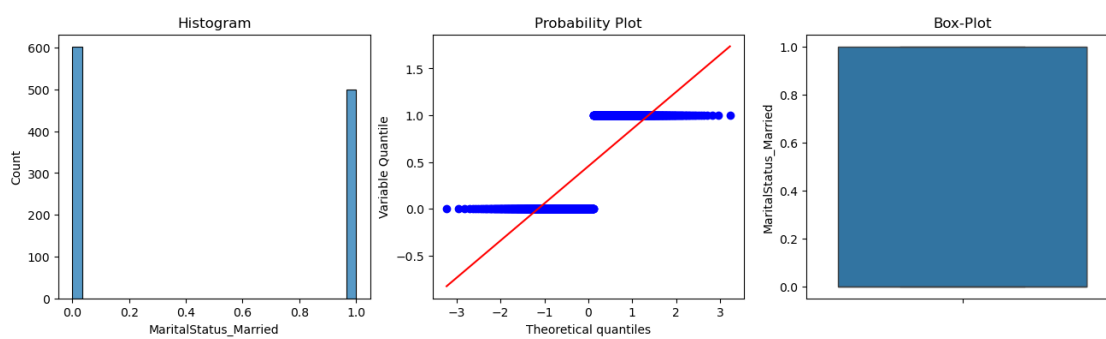












2.6 Outliers handling

```
[30]: winsorizer = Winsorizer(  
        capping_method = 'gaussian',  
        tail = 'both',  
        fold = 2,  
        variables = continuous  
    )  
    winsorizer.fit(tmp_mean_train)  
  
    win_train = winsorizer.transform(tmp_mean_train)  
    win_test = winsorizer.transform(tmp_mean_test)
```